REPUBLIC OF THE PHILIPPINES
BICOL UNIVERSITY
POLANGUI
COMPUTER STUDIES DEPARTMENT
Polangui, Albay

ISO 9001: 2015
SOCOTEC SCP000722Q

BAGONG PILIPINAS

WEB SYSTEMS AND TECHNOLOGIES

Phase 6: Update and Delete Functionalities

LAB REPORT

Uniformity: An E-Commerce platform for Pre-Loved School and Professional Uniforms

GROUP NAME: TechNic **Course & Block:** BSIT-2A		
MEMBERS	**ROLE**	**SIGNATURE**
Cipriano, Mcxyron B.	*Back-End Developer*	
Hernandez, Kurt Jushua S.	*Database Manager/Tester & Debugger*	
Registrado, Mharie Franz	*Tester & Debugger*	
Red, Paul Orlando B.	*Project, Database, GitHub Manager & Documentation Officer*	
Romano, Jay L.	*Front-End Developer*	

Submitted to:

Guillermo V. Red, Jr., DIT

Assistant Professor IV

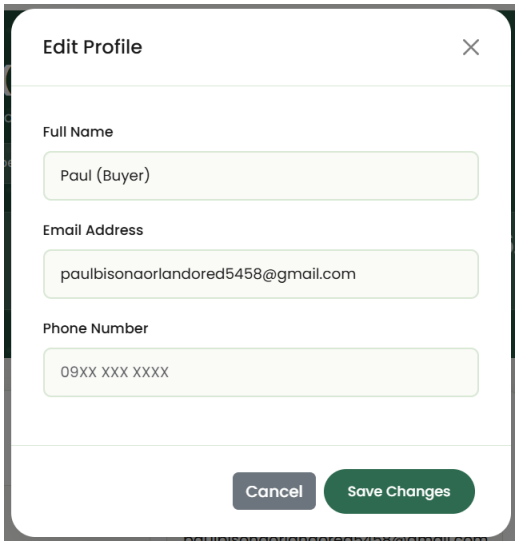
I. Overview

This lab report documents Phase 6 of our Uniformity project — **Update and Delete Functionalities.** Building on the data retrieval and display work completed in Phase 5, this phase focuses on completing the CRUD cycle by connecting frontend update and delete actions to the backend controllers, REST API routes, and MongoDB database operations.

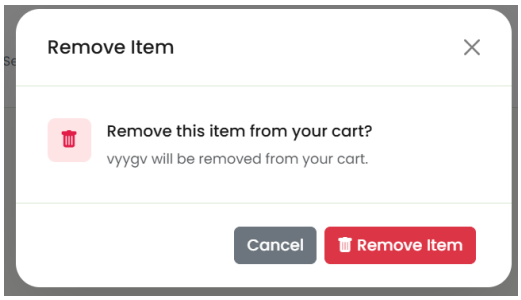
The primary deliverables of this phase include: **(1)** editable product listings through a working update modal; **(2)** deletion confirmation modals for safer listing removal; **(3)** backend update and delete routes connected to the database; **(4)** improved seller listing management with order details and fulfillment controls; **(5)** updated buyer order interactions, including order status confirmation; and **(6)** user interface fixes for loading states, badges, sold-out indicators, contact details, and layout consistency across the application.

II. Screenshots of working edit modal or page

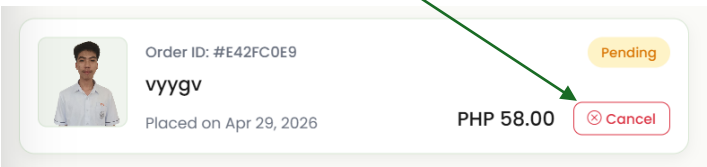
Edit Profile modal



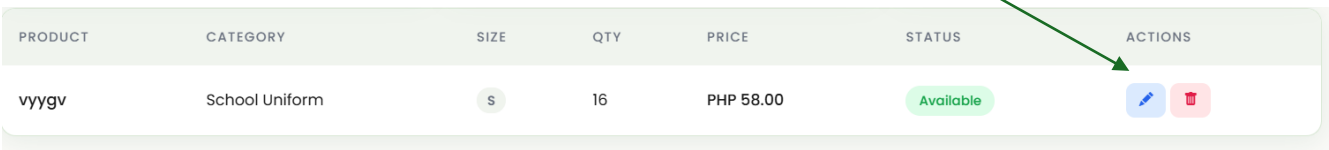
Modal confirmation if user wants to remove an item in cart

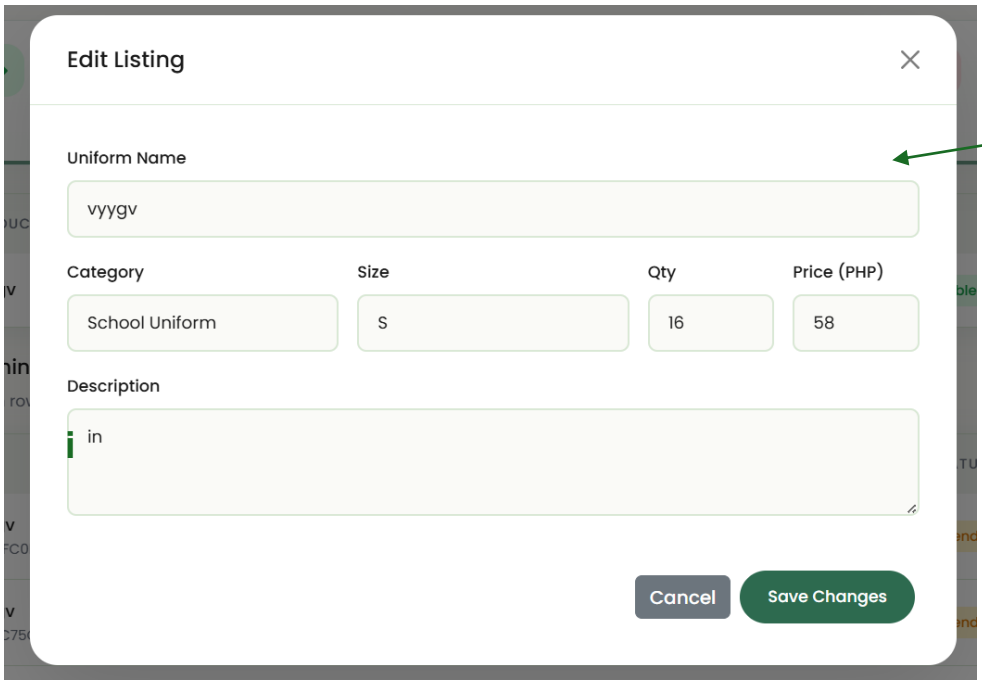


Cancel order button



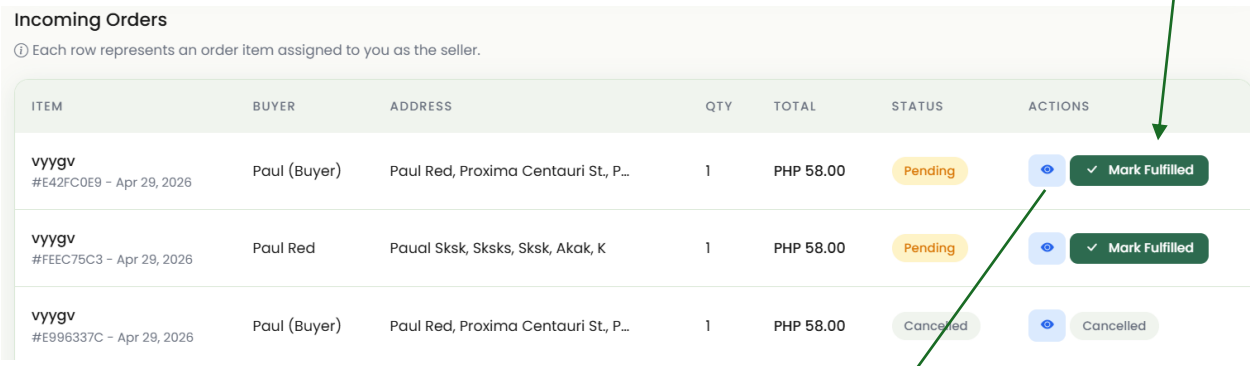
Update and delete listing button

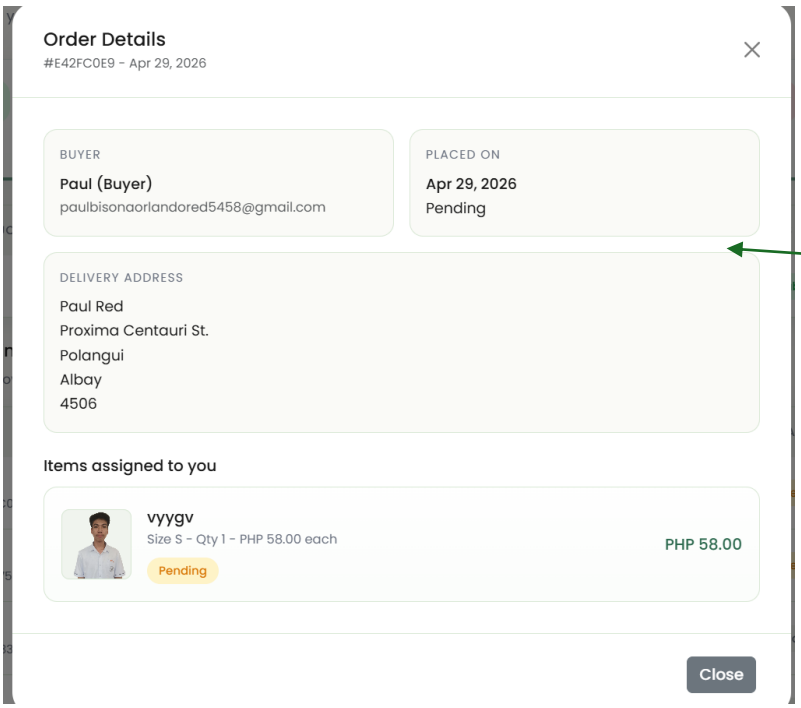




Edit listing modal: update name, category, size, quantity, price, and description. *(updating image is planned to implement in next phase for refinement)*

Update order status button

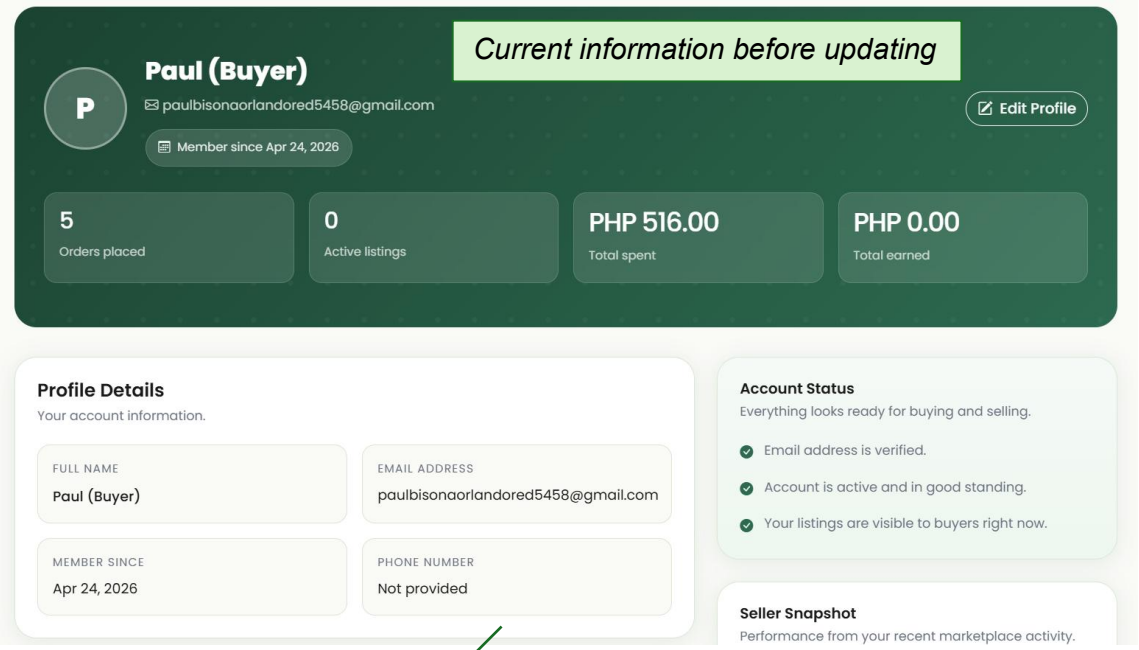




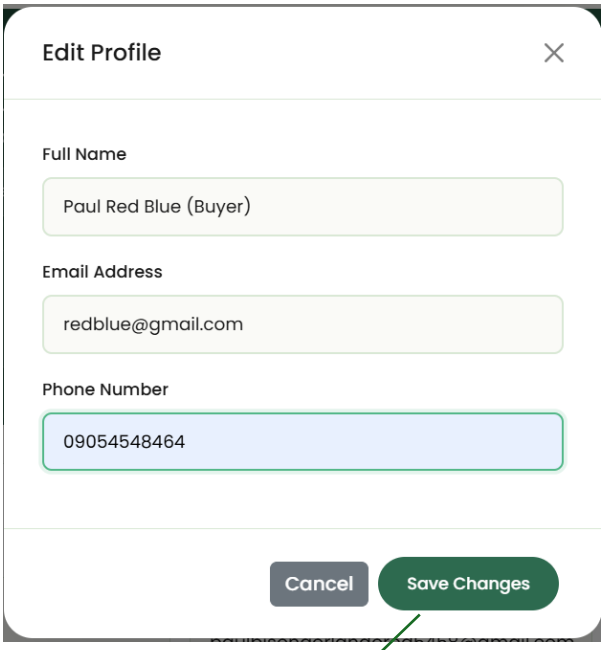
View order details modal of the buyer which includes necessary information for shipping out the item

III. Screenshots of update/ deletion confirmation and database updates

A. Edit Profile Modal



Editing the user profile



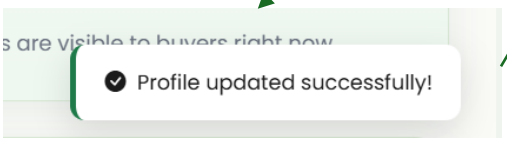
Current database

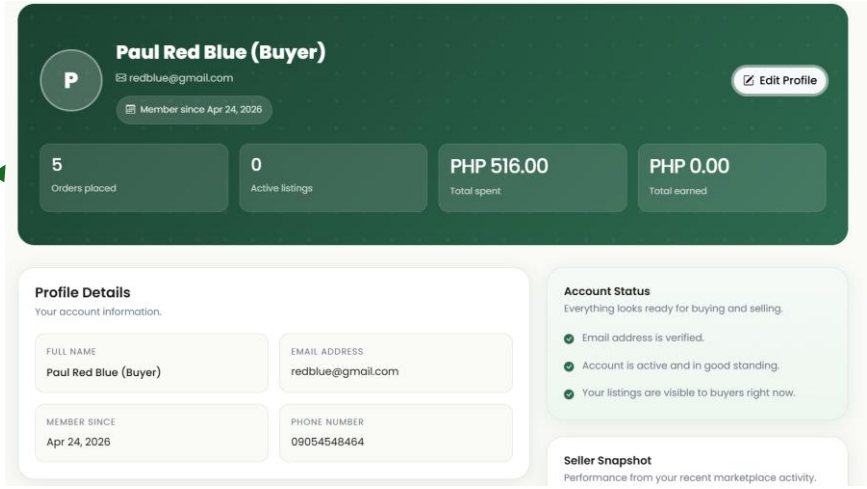

```
_id: ObjectId('69eb650f85d7262f0cfac8e2')
name : "Paul (Buyer)"
email : "paulbisonaorlandored5458@gmail.com"
password : "$2b$10$e0hUKYD.edPyp60LiVXjJeMOIYHC6KivTTQC8n9rhtIBTMiuSWzMO"
created_at : 2026-04-24T12:41:51.137+00:00
__v : 0
```


Updated database & Profile page


```
_id: ObjectId('69eb650f85d7262f0cfac8e2')
name : "Paul Red Blue (Buyer)"
email : "redblue@gmail.com"
password : "$2b$10$e0hUKYD.edPyp60LiVXjJeMOIYHC6KivTTQC8n9rhtIBTMiuSWzMO"
created_at : 2026-04-24T12:41:51.137+00:00
__v : 0
phone : "09054548464"
```

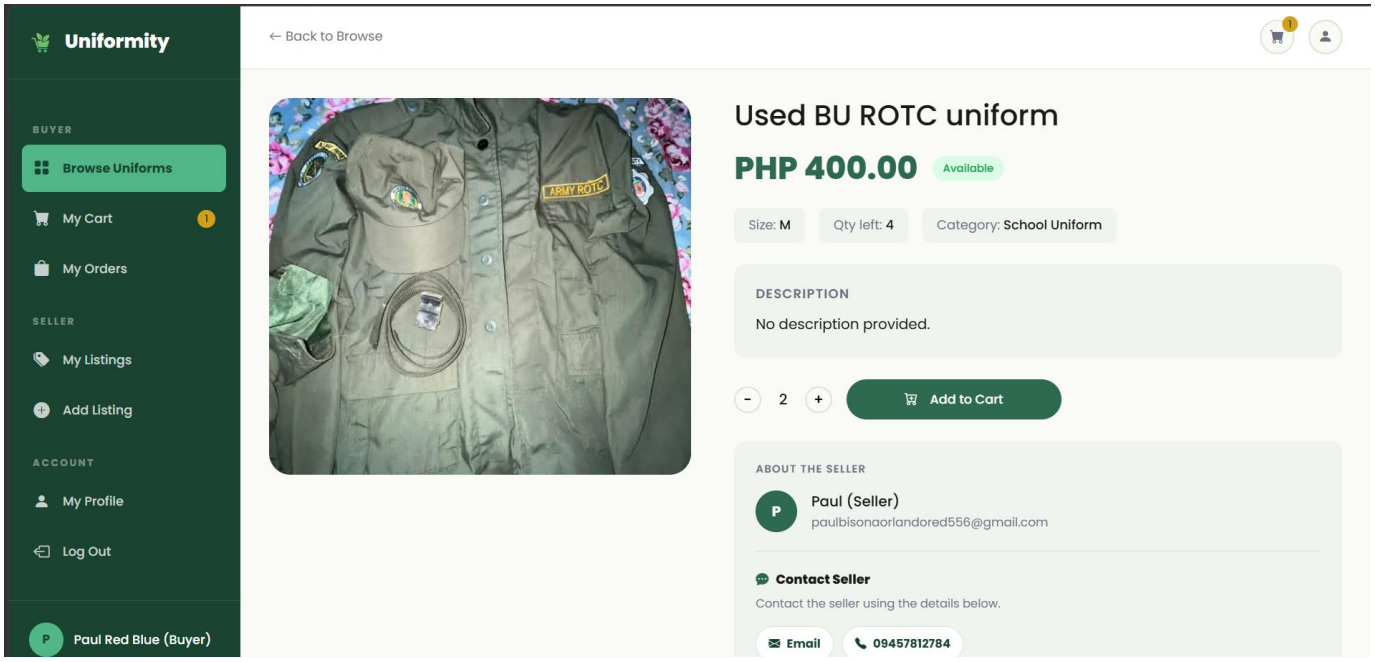

Updated confirmation





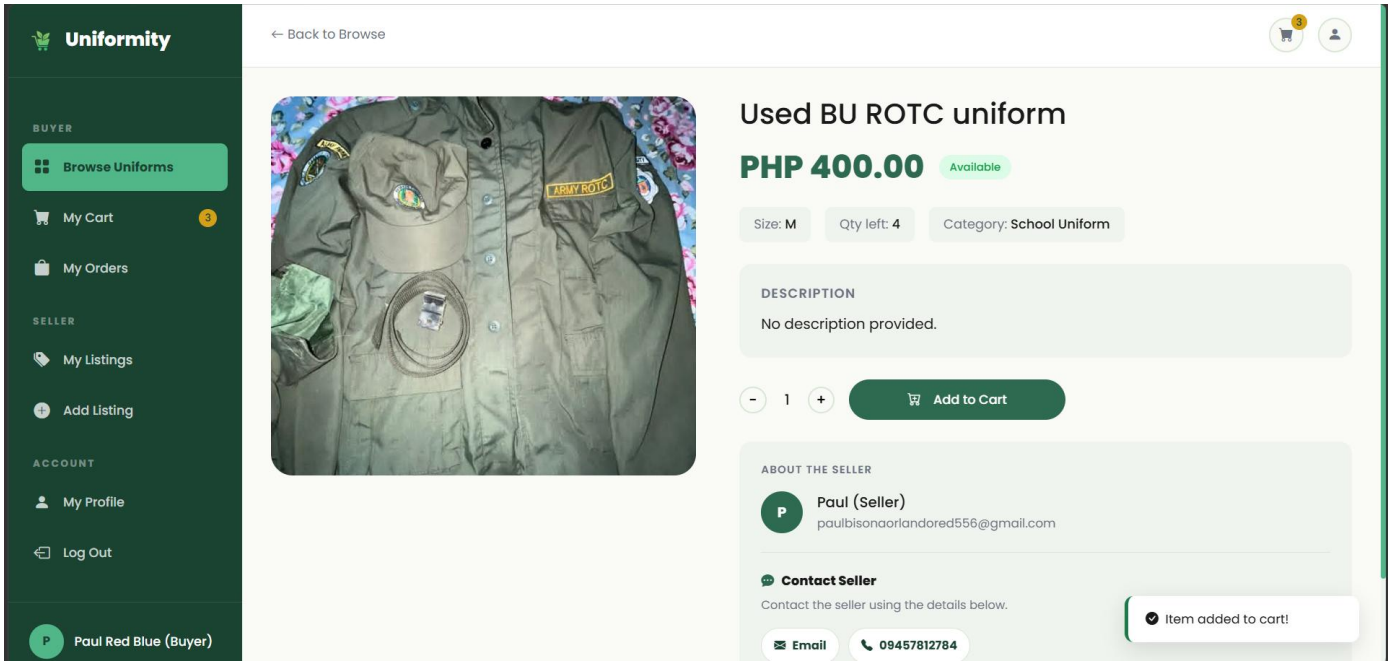
B. Adding *(update)* & removing *(delete)* an item in cart

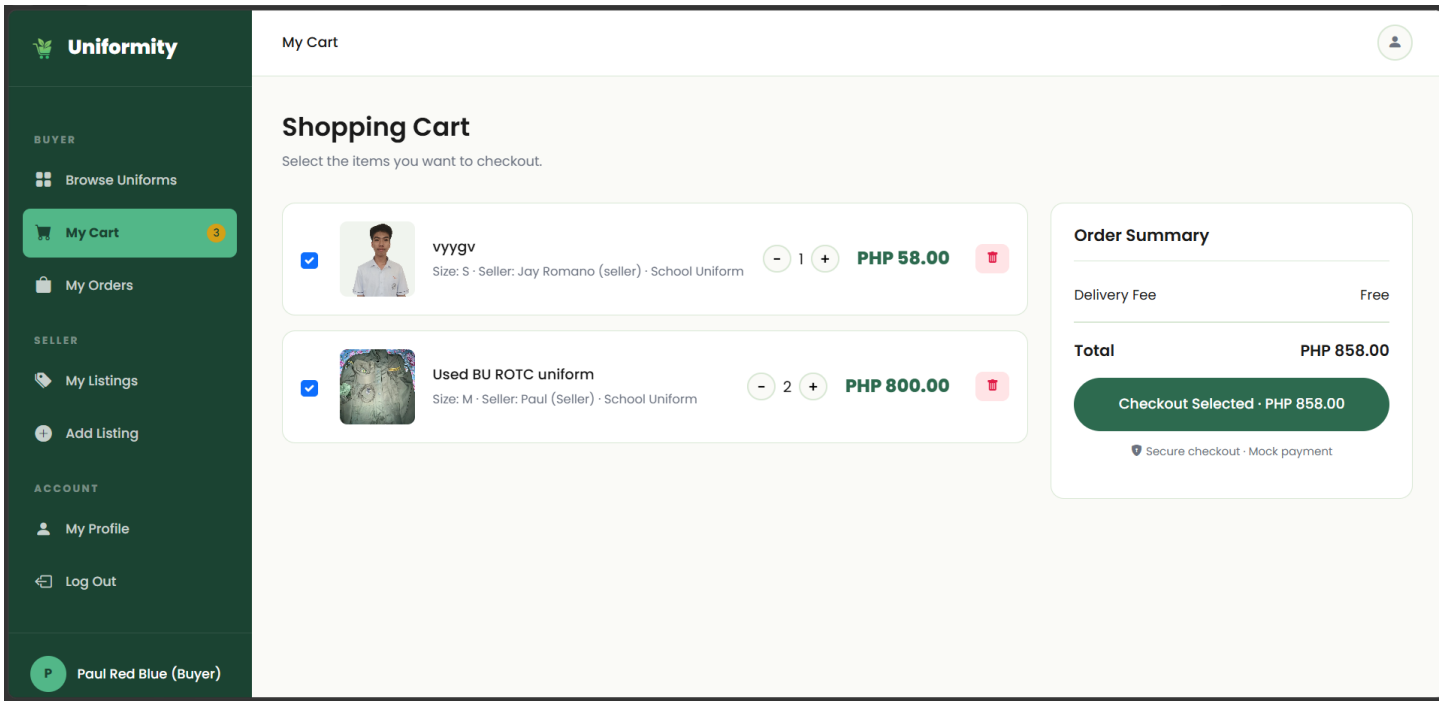
Currently the cart only has one item added with only 1 data in cartitems table




```
_id: ObjectId('69f22a10d48cfbfae42fc0f0')
cart_id : ObjectId('69f229dfd48cfbfae42fc0ee')
product_id : ObjectId('69f1f4850cadf5af9795c54a')
quantity : 1
__v : 0
```

After adding two items, the cart number badge updated to 3 with “Item added to cart confirmation” and also the database populated with another **product id** in the same **cart id** of the user



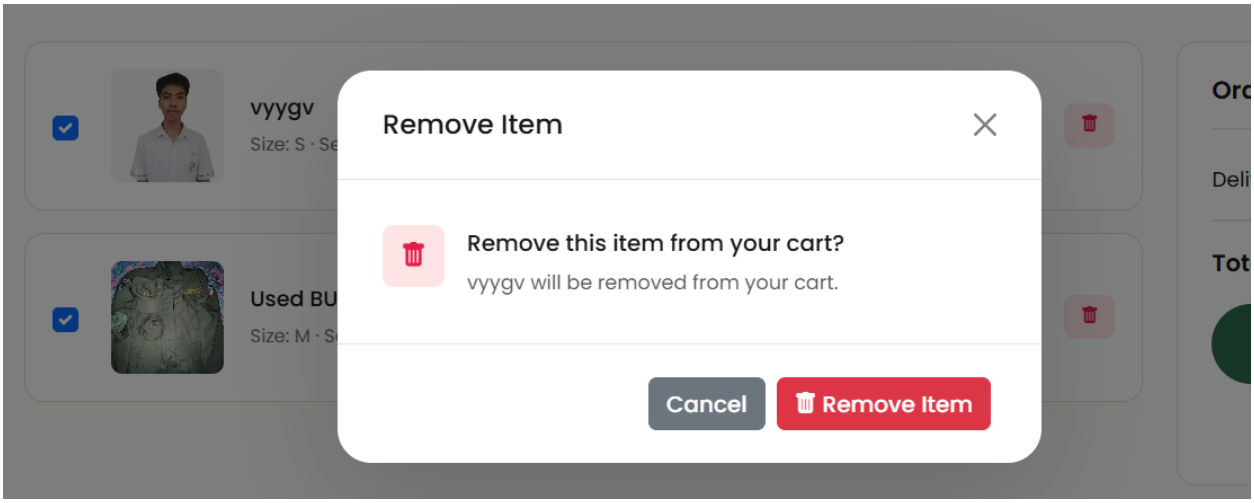


- ✓ **Database populated** after adding item with the same **cartid** of the user quantity of the added product

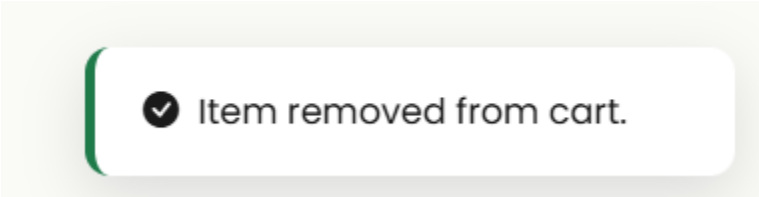
```
_id: ObjectId('69f22a10d48cfbfae42fc0f0')
cart_id : ObjectId('69f229dfd48cfbfae42fc0ee')
product_id : ObjectId('69f1f4850cadf5af9795c54a')
quantity : 1
__v : 0
```

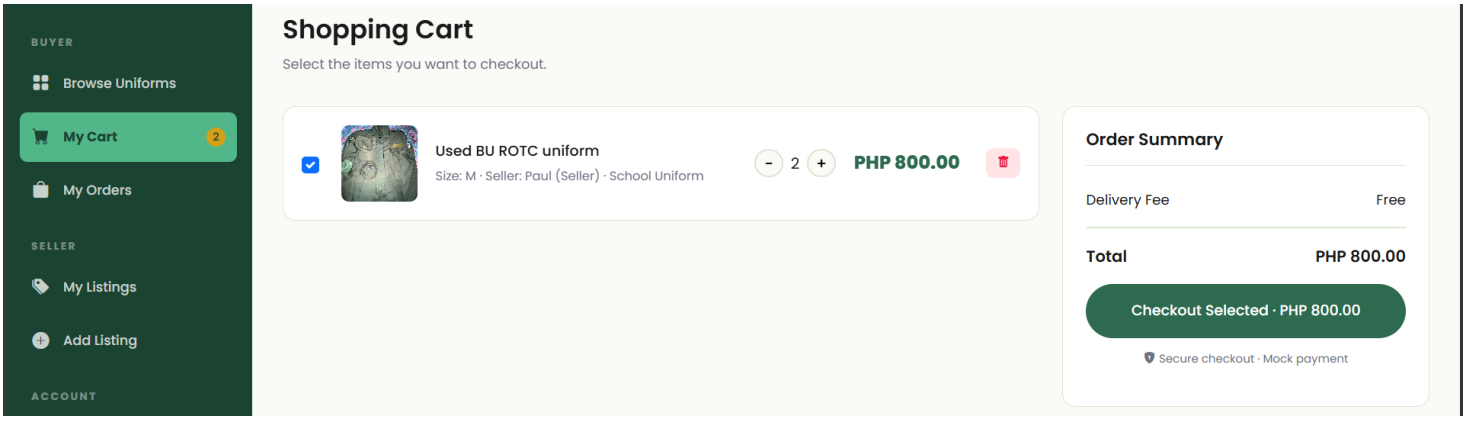
```
_id: ObjectId('69f22a74d48cfbfae42fc0f1')
cart_id : ObjectId('69f229dfd48cfbfae42fc0ee')
product_id : ObjectId('69ec5b1fce7f393ce8b3d895')
quantity : 2
__v : 0
```

Deleting an item in cart:

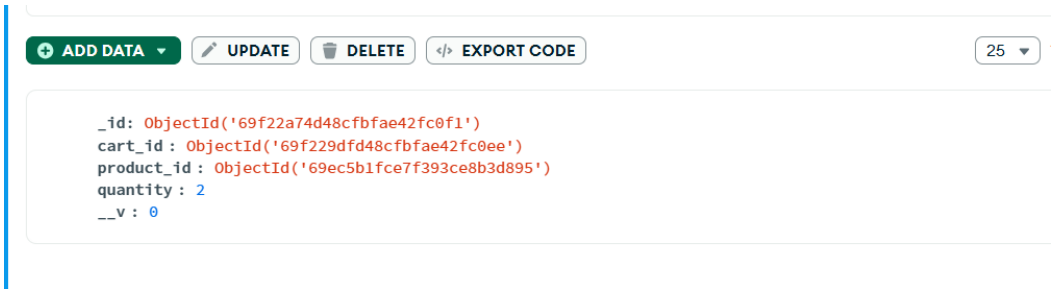


✓ Confirmation notification:

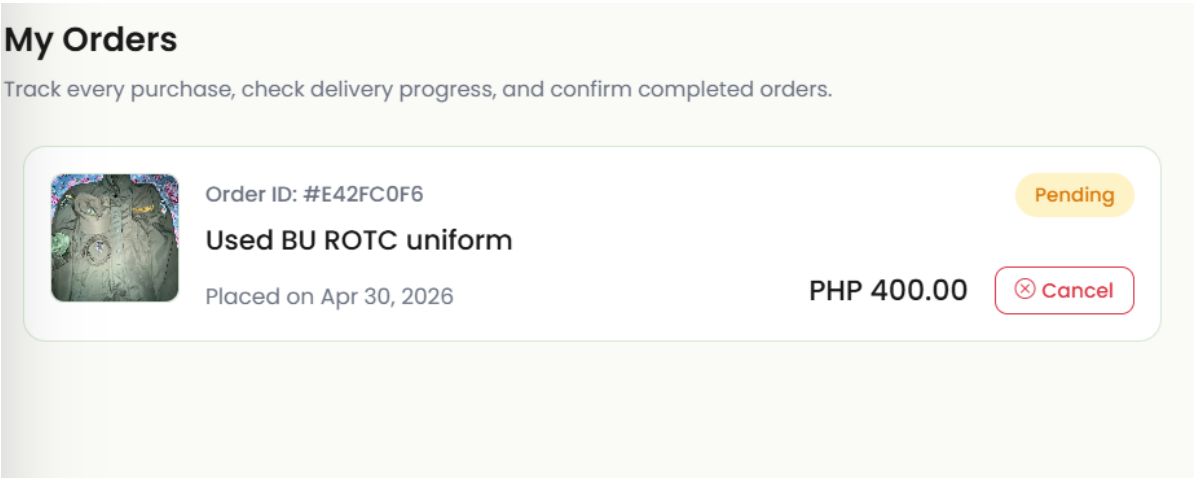




✓ Updated database data:

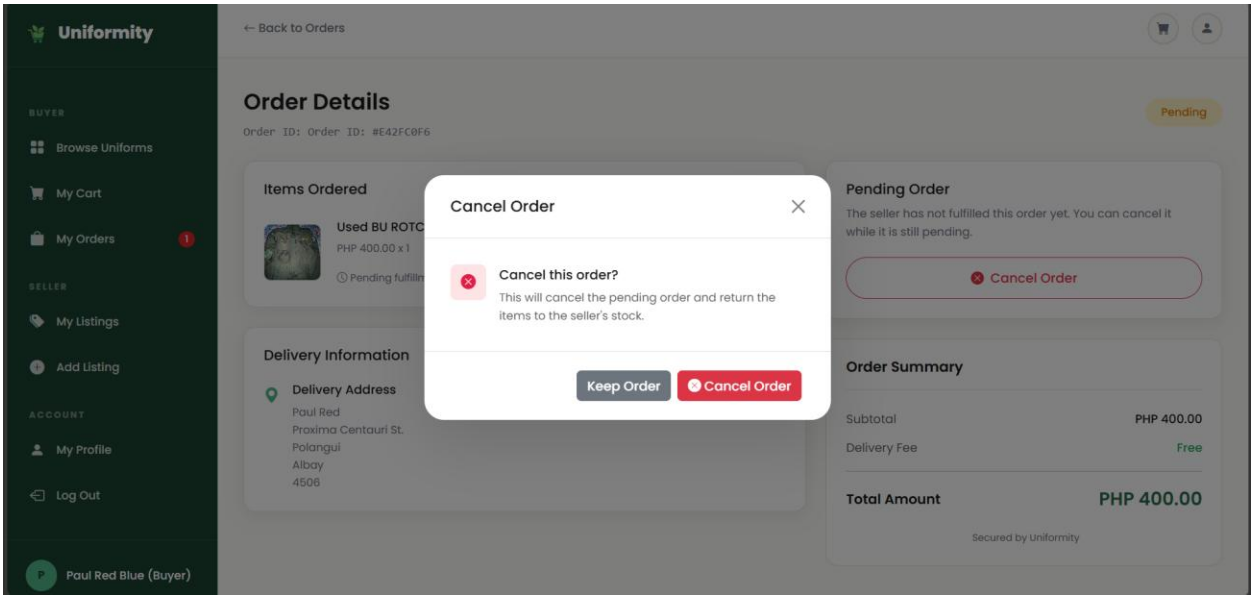


C. Cancelling an order

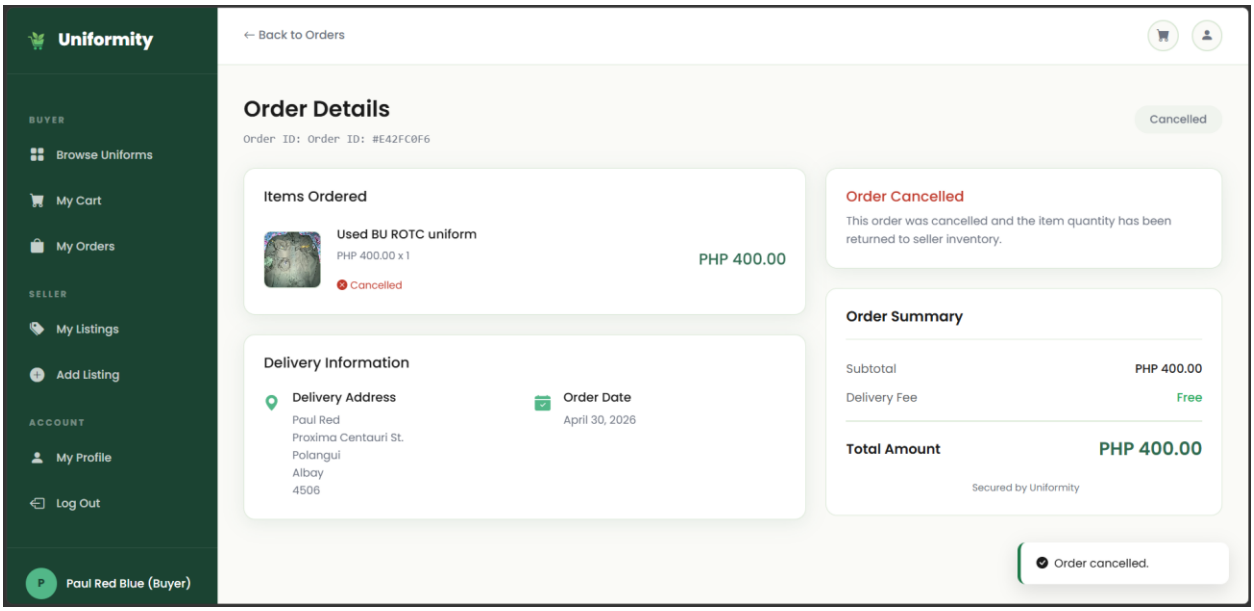


```
_id: ObjectId('69f22d44d48cfbfae42fc0f6')
buyer_id : ObjectId('69eb650f85d7262f0cfac8e2')
delivery_address : "Paul Red, Proxima Centauri St., Polangui, Albay, 4506"
total_amount : 400
status : "pending"
created_at : 2026-04-29T16:09:40.036+00:00
__v : 0
```

Cancellation confirmation modal



Cancelled **update display** and **notification**




```
_id: ObjectId('69f22d44d48cfbfae42fc0f6')
buyer_id : ObjectId('69eb650f85d7262f0cfac8e2')
delivery_address : "Paul Red, Proxima Centauri St., Polangui, Albay, 4506"
total_amount : 400
status : "cancelled"
created_at : 2026-04-29T16:09:40.036+00:00
__v : 0
```

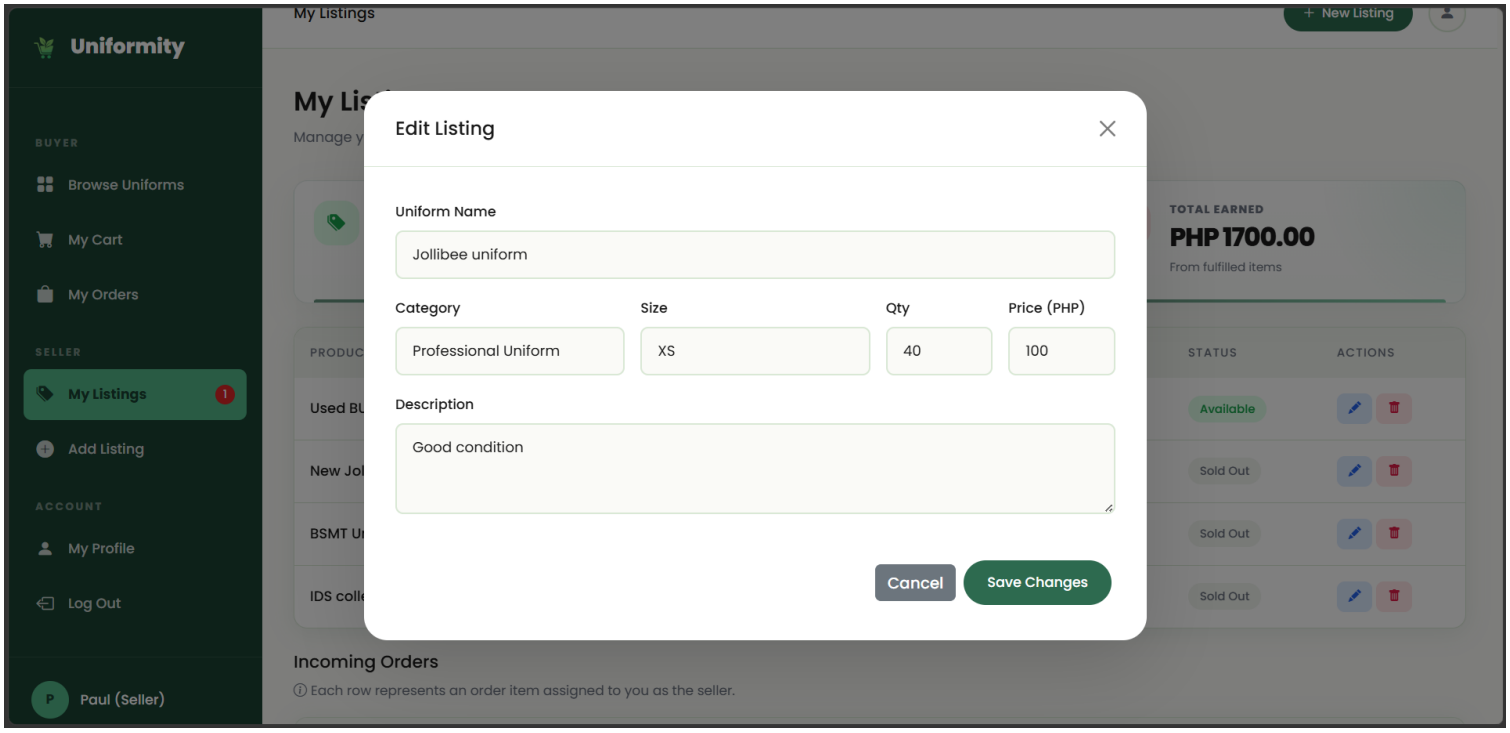
✓ Database order **updated to cancelled**

D. Editing *(update)* & removing *(delete)* listing

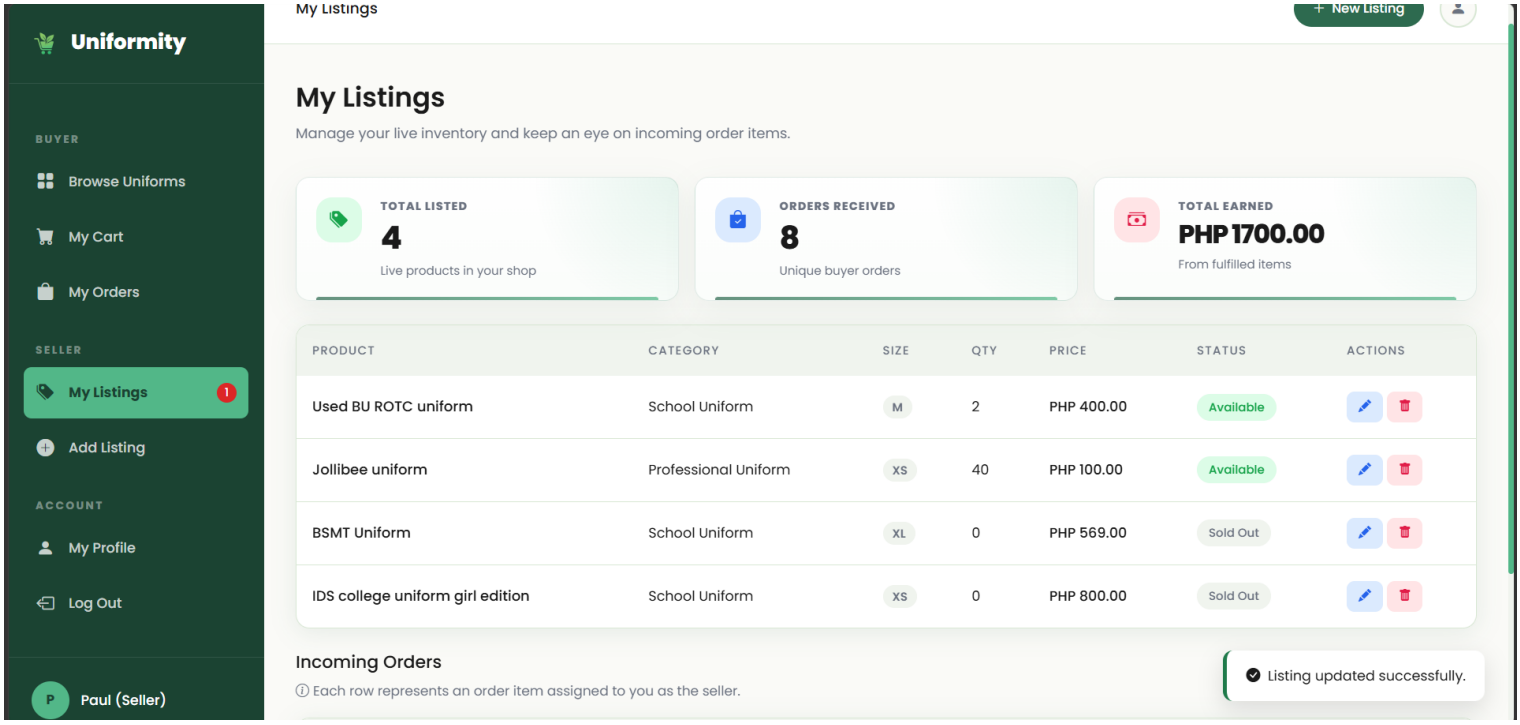
PRODUCT	CATEGORY	SIZE	QTY	PRICE	STATUS	ACTIONS
Used BU ROTC uniform	School Uniform	M	2	PHP 400.00	Available	
New Jollibee uniform	Professional Uniform	M	0	PHP 567.00	Sold Out	
BSMT Uniform	School Uniform	XL	0	PHP 569.00	Sold Out	
IDS college uniform girl edition	School Uniform	XS	0	PHP 800.00	Sold Out	


```
_id: ObjectId('69ec5c4cce7f393ce8b3d899')
name : "New Jollibee uniform"
size : "M"
quantity : 0
price : 567
description : "Good condition with 2 caps, nag resign na"
image_url : "https://res.cloudinary.com/dbx0kk6wq/image/upload/v1777097802/uniformi…"
seller_id : ObjectId('69eb659d85d7262f0cfac8e4')
category_id : ObjectId('69e040e0b6844662221fe952')
created_at : 2026-04-25T06:16:44.579+00:00
__v : 0
```


Editing the listing in the modal:



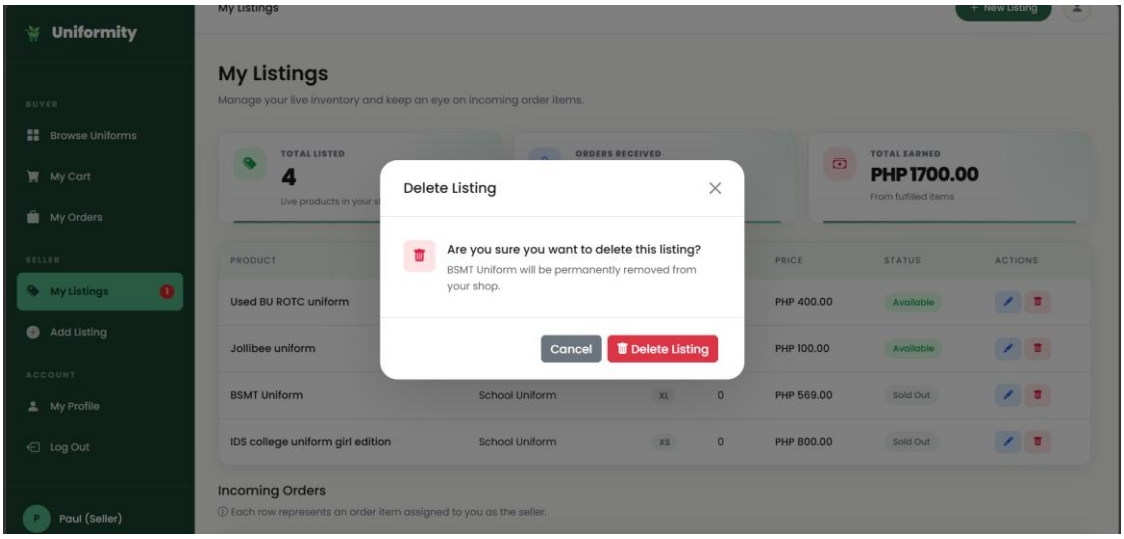
- ✓ Updated the changes made to the listing: Name, size, quantity, price, and status to available.
- ✓ Confirmation notification



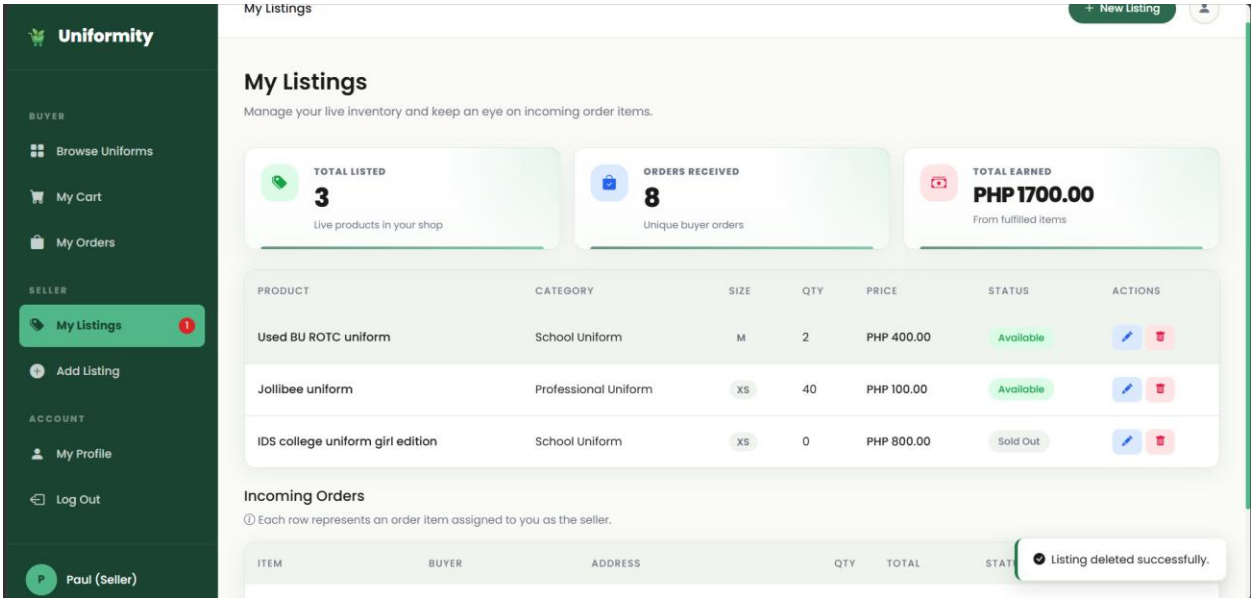
- ✓ Updated database

```
_id: ObjectId('69ec5c4cce7f393ce8b3d899')
name : "Jollibee uniform"
size : "XS"
quantity : 40
price : 100
description : "Good condition"
image_url : "https://res.cloudinary.com/dbx0kk6wq/image/upload/v1777097802/uniformi…"
seller_id : ObjectId('69eb659d85d7262f0cfac8e4')
category_id : ObjectId('69e040e0b6844662221fe952')
created_at : 2026-04-25T06:16:44.579+00:00
__v : 0
```

Deleting a listing: *ex. BSMT Uniform*

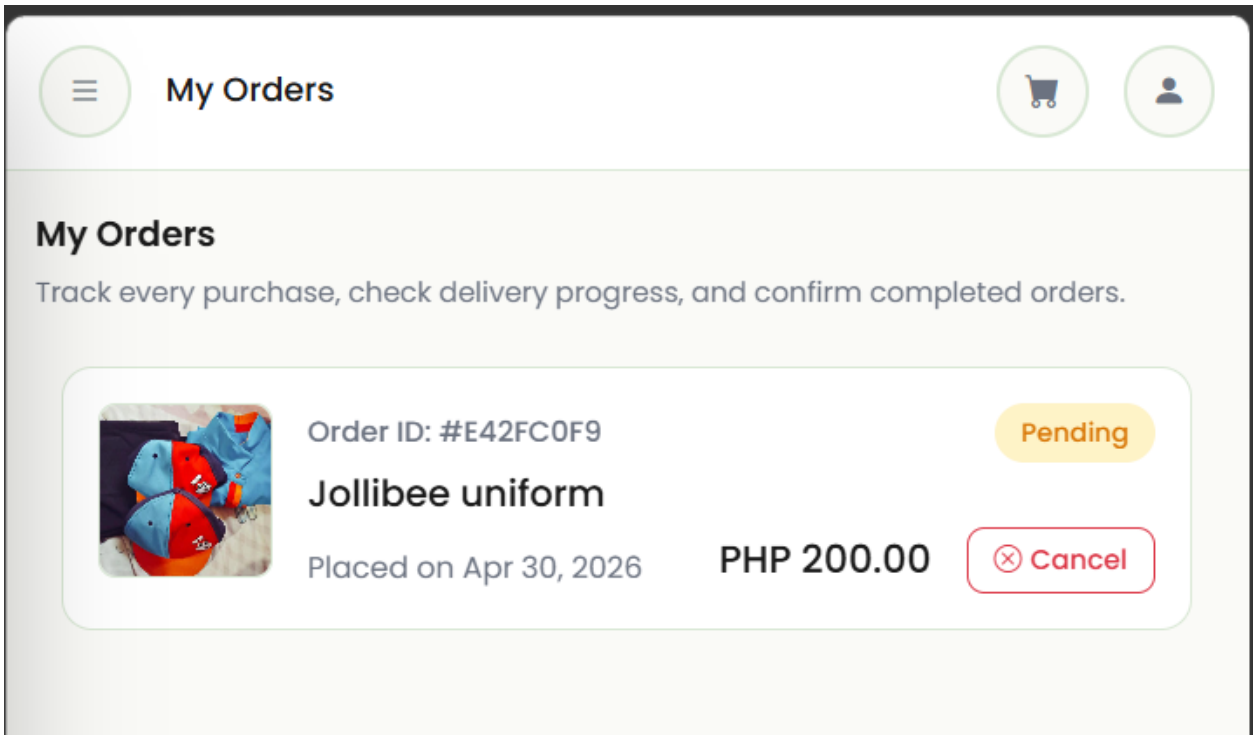


✓ *BSMT Uniform* deleted in the listing



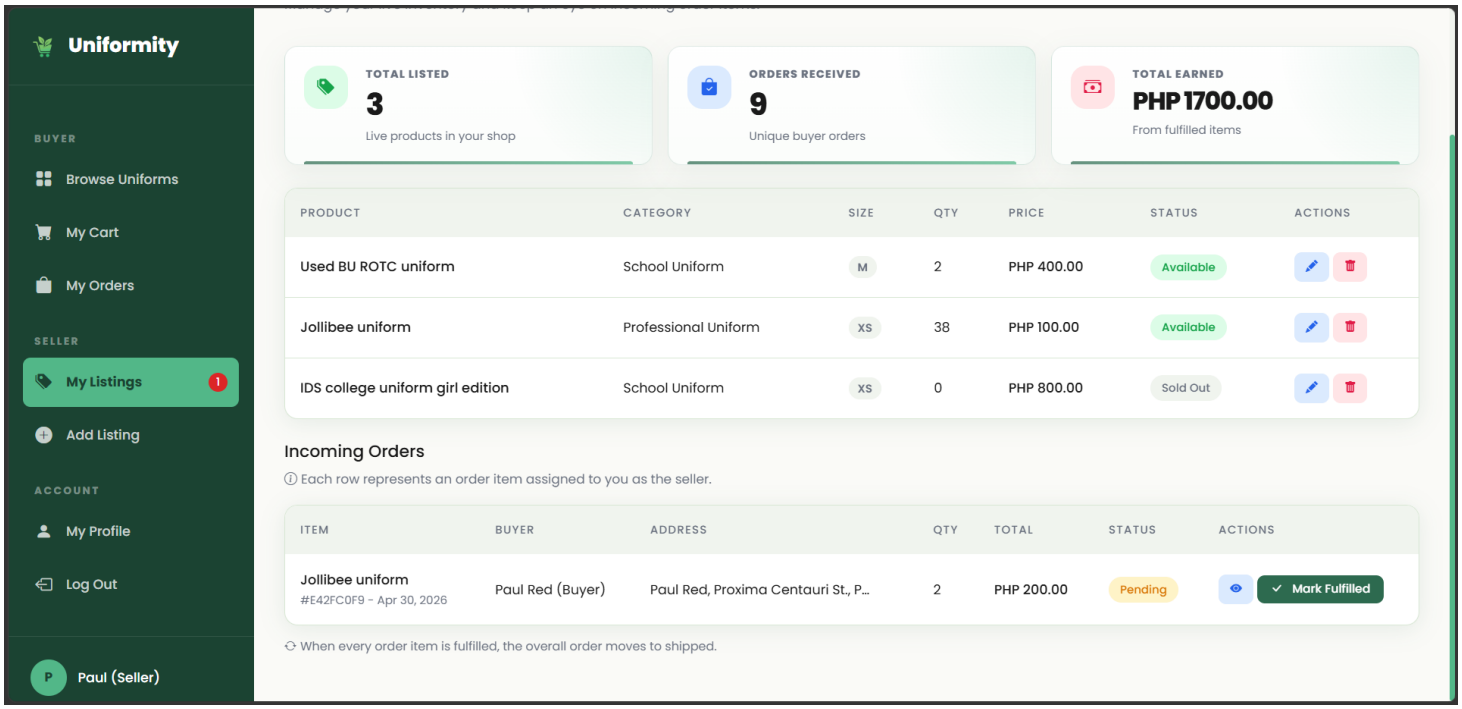
E. Updating order status *(mark as fulfilled and mark as received)*

After placing an order: ***(buyer's view)***



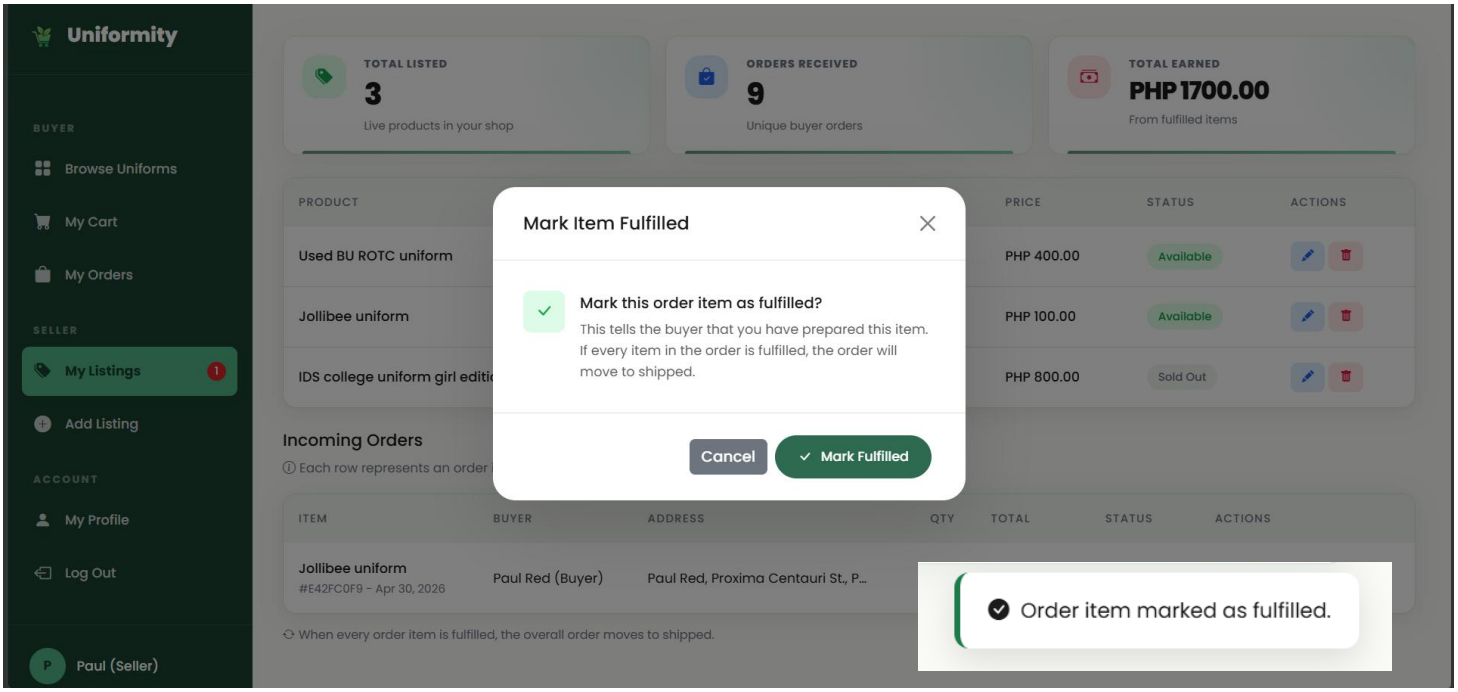
After placing an order: ***(seller's view)***

- ✓ The order *Jollibee uniform* is posted and status is in pending

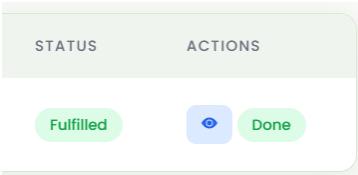


Marking *(update)* the order as fulfilled

- The order should be already prepared and shipped

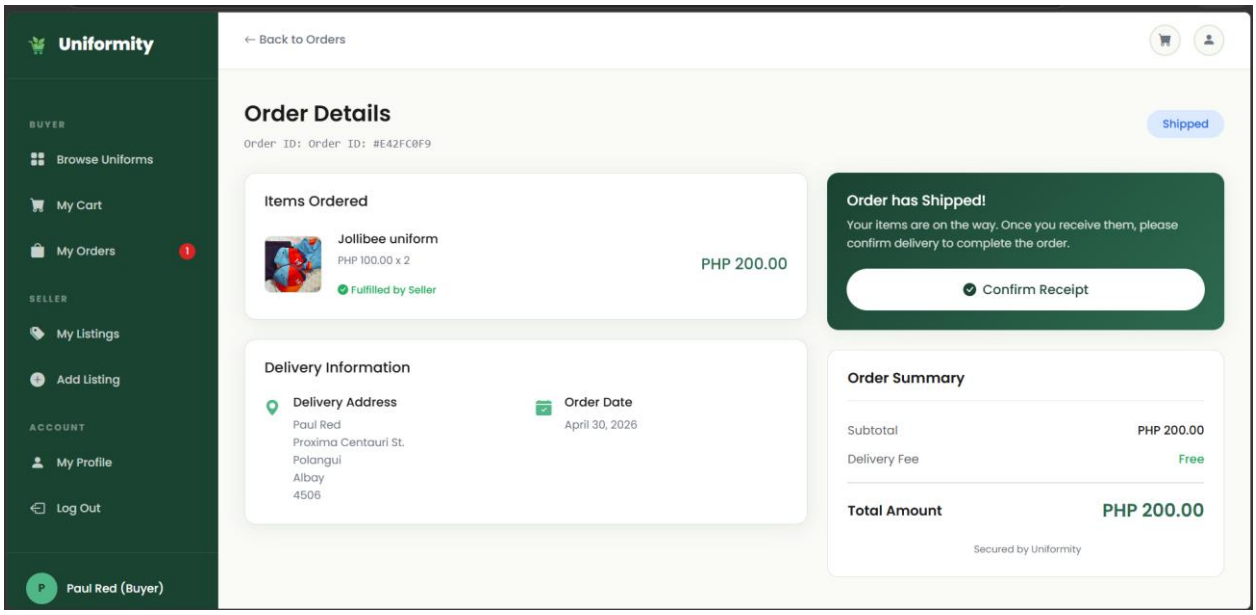


- ✓ Notification confirmation will appear
- ✓ Update the status to **fulfilled**
- ✓ Database status updated to **shipped**

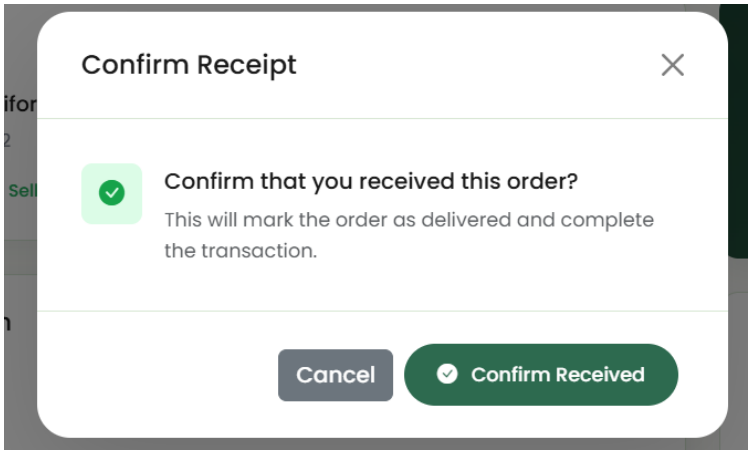



```
_id: ObjectId('69f233c5d48cfbfae42fc0f9')
buyer_id : ObjectId('69eb650f85d7262f0cfac8e2')
delivery_address : "Paul Red, Proxima Centauri St., Polangui, Albay, 4506"
total_amount : 200
status : "shipped"
created_at : 2026-04-29T16:37:25.455+00:00
__v : 0
```

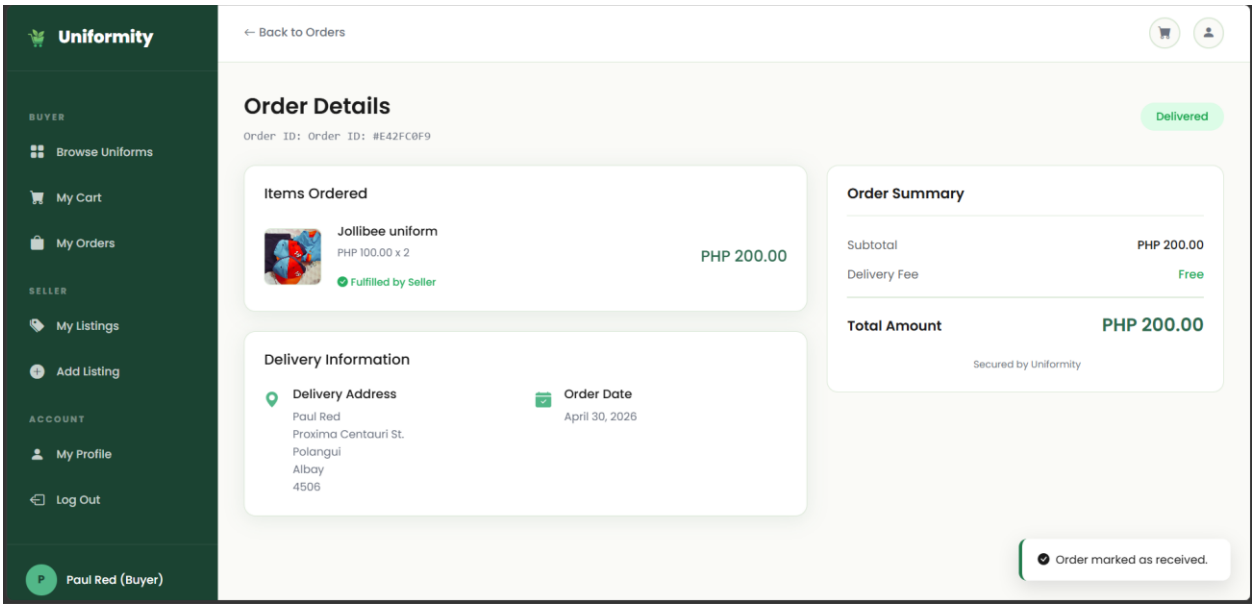
In buyer's end, the order status is set to shipped and ready to **confirm receipt** button is active



Confirmation modal before confirm received:



All status changed to delivered and notification confirmation is added



IV. JavaScript snippets for edit and delete

Edit listing

```
async function submitListingUpdate(event) {
    event.preventDefault();

    const token = localStorage.getItem('token');
    const form = event.currentTarget;
    if (!token || !editingProductId || !form) return;

    const btn = form.querySelector('button[type="submit"]');
    const originalText = btn?.innerHTML;

    const payload = {
        name: form.elements.name.value.trim(),
        category_id: form.elements.category_id.value,
        size: form.elements.size.value,
        quantity: Number(form.elements.quantity.value),
        price: Number(form.elements.price.value),
        description: form.elements.description.value.trim(),
    };

    if (!payload.name || !payload.category_id || !payload.size || payload.quantity < 0 || payload.price < 1) {
        showToast('Please complete the listing details before saving.', 'error');
        return;
    }
```

Delete listing

```
async function deleteSelectedListing() {
    const token = localStorage.getItem('token');
    if (!token || !deletingProductId) return;

    const btn = document.getElementById('confirmDeleteListingBtn');

    const originalText = btn?.innerHTML;
    if (btn) {
        btn.disabled = true;
        btn.innerHTML = '<span class="spinner-border spinner-border-sm me-2"></span>Deleting...';
    }

    try {
        const res = await fetch(`${API_BASE}/products/${deletingProductId}`, {
            method: 'DELETE',
            headers: { 'x-auth-token': token }
        });
        const data = await res.json();
        if (!res.ok) throw new Error(data.error || data.msg || 'Could not delete listing.');

        const modal = bootstrap.Modal.getInstance(document.getElementById('deleteListingModal'));
        if (modal) modal.hide();
```

```
        deletingProductId = null;
        showToast('Listing deleted successfully.');
        await loadMyListings();
        await loadSellerStats();
    } catch (err) {
        showToast(err.message || 'Could not delete listing.', 'error');
    } finally {
        if (btn) {
            btn.disabled = false;
            btn.innerHTML = originalText;
        }
    }
```

Deleting from cart

```
async function submitCartRemoval() {
    if (!pendingCartRemoval) return;

    const { itemId, btn } = pendingCartRemoval;
    const confirmBtn = document.getElementById('confirmDeleteCartItemBtn');
    const originalConfirmText = confirmBtn?.innerHTML;

    if (confirmBtn) {
        confirmBtn.disabled = true;
        confirmBtn.innerHTML = '<span class="spinner-border spinner-border-sm me-2"></span>Removing...';
    }

    const removed = await removeFromCart(itemId, btn);

    if (removed) {
        const modal = bootstrap.Modal.getInstance(document.getElementById('deleteCartItemModal'));
        if (modal) modal.hide();
        pendingCartRemoval = null;
    }

    if (confirmBtn) {
        confirmBtn.disabled = false;
        confirmBtn.innerHTML = originalConfirmText;
```

Edit Profile

```
const editProfileForm = document.getElementById('editProfileForm');
if (editProfileForm) {
    editProfileForm.addEventListener('submit', async (e) => {
        e.preventDefault();
        clearAllErrors(editProfileForm);

        const token = getToken();
        const user = getUser();

        if (!token || !user) {
            showToast('You must be logged in.', 'error');
            return;
        }

        const nameInput = editProfileForm.querySelector('[name="name"]');
        const emailInput = editProfileForm.querySelector('[name="email"]');
        const phoneInput = editProfileForm.querySelector('[name="phone"]');
        const btn = editProfileForm.querySelector('button[type="submit"]');
        const originalText = btn.innerHTML;
```

Update order to fulfilled

```
async function submitListingUpdate(event) {
    try {
        const res = await fetch(`${API_BASE}/products/${editingProductId}`, {
            method: 'PUT',
            headers: {
                'Content-Type': 'application/json',
                'x-auth-token': token
            },
            body: JSON.stringify(payload)
        });
        const data = await res.json();
        if (!res.ok) throw new Error(data.error || data.msg || 'Could not update listing.');

        const modal = bootstrap.Modal.getInstance(document.getElementById('editModal'));
        if (modal) modal.hide();

        editingProductId = null;
        showToast('Listing updated successfully.');
        await loadMyListings();
        await loadSellerStats();
    } catch (err) {
```

Update order to confirm receipt (delivered)

```
function confirmReceipt(orderId, btn) {
    pendingReceiptConfirmation = { orderId, btn };

    const modalEl = document.getElementById('confirmReceiptModal');
    if (!modalEl) return;

    const modal = new bootstrap.Modal(modalEl);
    modal.show();
}

async function submitReceiptConfirmation() {
    const token = localStorage.getItem('token');
    if (!token || !pendingReceiptConfirmation) return;

    const { orderId, btn } = pendingReceiptConfirmation;
    const confirmBtn = document.getElementById('confirmReceiptBtn');
    const originalText = btn?.innerHTML;
    const originalConfirmText = confirmBtn?.innerHTML;
```

Canceling order

```
async function submitCancelConfirmation() {
    const token = localStorage.getItem('token');
    if (!token || !pendingCancelConfirmation) return;

    const { orderId, btn } = pendingCancelConfirmation;
    const confirmBtn = document.getElementById('confirmCancelOrderBtn');
    const originalText = btn?.innerHTML;
    const originalConfirmText = confirmBtn?.innerHTML;

    if (btn) {
        btn.disabled = true;
        btn.innerHTML = '<span class="spinner-border spinner-border-sm me-1"></span>Cancelling...';
    }
    if (confirmBtn) {
        confirmBtn.disabled = true;
        confirmBtn.innerHTML = '<span class="spinner-border spinner-border-sm me-2"></span>Cancelling...';
    }

    try {
        const res = await fetch(`${API_BASE}/orders/${orderId}/cancel`, {
            method: 'PATCH',
            headers: { 'x-auth-token': token }
```

V. Backend route and controller snippets

✓ Update profile

```
router.put('/:id', auth, updateUser);
```

```
    const user = await User.findByIdAndUpdate(req.params.id, updates, { new: true, runValidators: true }).select('-password');
    res.json(user);
} catch (err) {
    res.status(400).json({ error: err.message });
}
```

✓ Update product listing

```
router.put('/:id', [auth, upload.single('image')], updateProduct);
```

```
exports.updateProduct = async (req, res) => {
    try {
        const product = await Product.findById(req.params.id);
        if (!product) return res.status(404).json({ error: 'Product not found' });

        if (String(product.seller_id) !== String(req.user.id)) {
            return res.status(403).json({ error: 'You can only update your own listing.' });
        }

        const updates = {
            name: req.body.name,
            category_id: req.body.category_id,
            size: req.body.size,
            price: req.body.price,
            quantity: req.body.quantity,
            description: req.body.description,
        };
```

✓ Mark seller order item as fulfilled

```
router.get('/seller/:sellerId', auth, getSellerOrders);
```

```
exports.getSellerOrders = async (req, res) => {
    try {
        if (String(req.user.id) !== String(req.params.sellerId)) {
            return res.status(403).json({ error: 'Access denied' });
        }

        const sellerItems = await OrderItem.find({ seller_id: req.params.sellerId })
            .populate({
                path: 'product_id',
                select: 'name price image_url size category_id seller_id',
                populate: [
                    { path: 'category_id', select: 'name' },
                    { path: 'seller_id', select: 'name email' },
                ],
            });
```

✓ Mark buyer order as delivered/received

```
20    router.get('/:orderId', auth, getOrderById);
```

```
exports.markAsDelivered = async (req, res) => {
    try {
        const order = await Order.findById(req.params.orderId);
        if (!order) return res.status(404).json({ error: 'Order not found' });

        if (String(order.buyer_id) !== String(req.user.id)) {
            return res.status(403).json({ error: 'You can only update your own order.' });
        }

        if (order.status !== 'shipped') {
            return res.status(400).json({ error: 'Only shipped orders can be marked as delivered.' });
        }

        order.status = 'delivered';
        await order.save();

        res.json({ message: 'Order marked as delivered' });
    } catch (err) {
        res.status(500).json({ error: err.message });
    }
```

✓ Delete product listing

```
// Protected update and delete
router.put('/:id', [auth, upload.single('image')], updateProduct);
router.delete('/:id', auth, deleteProduct);
```

```
exports.deleteProduct = async (req, res) => {
    try {
        const product = await Product.findById(req.params.id);
        if (!product) return res.status(404).json({ error: 'Product not found' });

        if (String(product.seller_id) !== String(req.user.id)) {
            return res.status(403).json({ error: 'You can only delete your own listing.' });
        }

        await Product.findByIdAndDelete(req.params.id);
        res.json({ message: 'Product deleted successfully' });
    } catch (err) {
        res.status(500).json({ error: err.message });
    }
};
```

✓ Remove item from cart

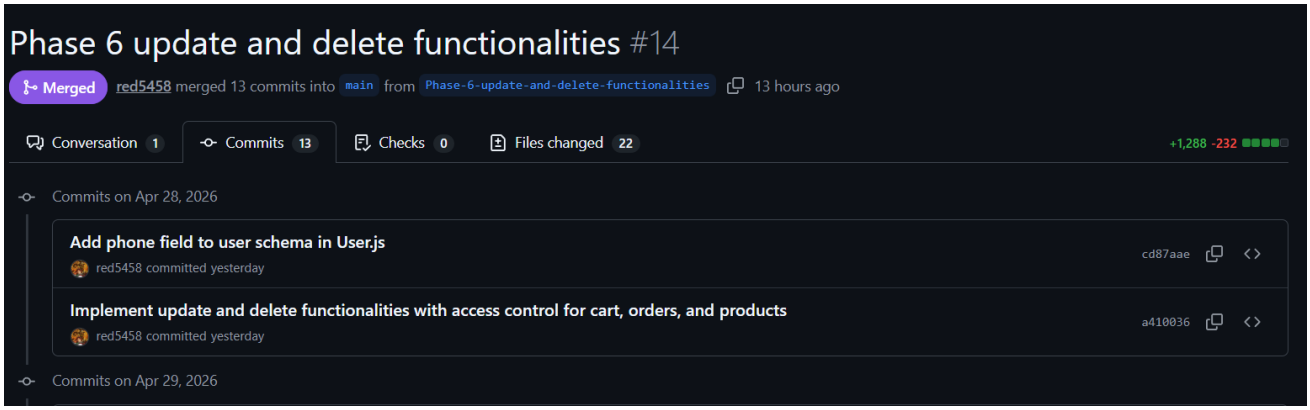
```
router.delete('/item/:itemId', auth, removeFromCart);
```

```
exports.removeFromCart = async (req, res) => {
    try {
        const item = await CartItem.findById(req.params.itemId);
        if (!item) {
            return res.status(404).json({ error: 'Cart item not found.' });
        }

        const cart = await Cart.findById(item.cart_id);
        if (!cart || cart.user_id.toString() !== req.user.id) {
            return res.status(403).json({ error: 'You can only update your own cart.' });
        }

        await item.deleteOne();
        res.json({ message: 'Item removed from cart' });
    } catch (err) {
        res.status(500).json({ error: err.message });
    }
};
```

VI. GitHub commit logs



Add phone field to edit profile form and update user data handling
Clyve117 committed 13 hours ago 6969084

Hide cart summary on load for unauthenticated users and adjust layout for empty cart state
Clyve117 committed 13 hours ago 82b8eec

Enhance product card functionality with stock status and sold out indication; update loading and error messages for better user experience
Clyve117 committed 13 hours ago 7a353c7

Add order count badges to sidebar and implement badge update logic
Clyve117 committed 13 hours ago 48748b2

Implement listing management features: add category loading, enhance order details, and improve UI for listings and orders
Clyve117 committed 13 hours ago 8457481

Implement receipt confirmation functionality and enhance order error handling
Clyve117 committed 13 hours ago 68135c0

Enhance order loading and receipt confirmation functionalities; update loading states and improve error handling
Clyve117 committed 13 hours ago 7486ac2

Enhance product detail rendering: add sold out indication, improve seller contact display, and update add to cart button logic
Clyve117 committed 13 hours ago 2f85764

Enhance profile population and recent orders loading: add phone number display, improve input field handling, and update loading/error messages styling
Clyve117 committed 13 hours ago b60f99d

Enhance seller statistics and product card styling: add responsive layouts, improve hover effects, and update disabled button states
Clyve117 committed 13 hours ago dd48167

Updated html enhance cart and orders pages: add loading states, improve UI for order details, and implement receipt confirmation modal
Clyve117 committed 13 hours ago be0fc8b

Update API_BASE to dynamically set the API URL based on the environment
red5458 committed 3 hours ago · ✓ 1 / 1

Fix formatting in package.json by adding missing newline at end of file
red5458 committed 3 hours ago · ✓ 1 / 1

Add development script to package.json for easier local testing
red5458 committed 3 hours ago · ✓ 1 / 1

Merge pull request #16 from red5458/feat-Add-cancel-order-function
red5458 authored 3 hours ago · ✓ 1 / 1

Implement cancel order feature with UI updates and status handling
red5458 committed 4 hours ago · ✓ 2 / 2

Add cancel order functionality with frontend and backend integration
red5458 committed 4 hours ago

Add confirmation modal for item removal from cart
red5458 committed 4 hours ago · ✓ 1 / 1

Cleanup typography weights for consistency across headings and text elements
red5458 committed 5 hours ago

Improve checkout summary rendering
red5458 committed 5 hours ago

VII. Short reflections from each group member (2–3 sentences)

- **Cipriano, Mcxyron B.** — Back-End Developer

In this phase, I got to learn more deeply about how update and delete functions work in the server side, especially using routes and controllers. My contribution was helping with the backend part like editing product data, deleting listings, and making sure the database updates properly. Our manager also guided us on what feature should be finished first, so it was easier for me to know what backend function I need to focus on.

- **Hernandez, Kurt Joshua S.** — Database Manager / Tester & Debugger

For this phase, testing the database is important because sometimes the frontend looks working but the data is not really changing correctly. As the testing happen, me and my fellow database manager, always checking in the database if the updated and deleted data are reflected in MongoDB. I also helped test the flow with the group, pointing out what pages still need checking.

- **Registrado, Mharie Franz** — Tester & Debugger

In this phase, I learned how important testing is because some errors only appear when we actually click the buttons and try different actions. My contribution was helping test the edit modal, delete confirmation, cart, orders, and other pages to see if there are bugs before we proceed to the next phase.

- **Red, Paul Orlando B.** — Project, Database, GitHub Manager & Documentation Officer

In this phase, I instruct my fellow members with each role on what they will be doing in this phase. My contribution was guiding the group on what tasks to do, checking the database, managing the GitHub updates (merging pull requests), and preparing the documentation for the lab report. I also helped test the features and made sure the screenshots, code snippets, and explanations are complete.

- **Romano, Jay L.** — Front-End Developer

For Phase 6, My contribution was working on the frontend part like the edit listing modal, delete confirmation modal, and making sure the page updates after editing or deleting. Project manager also helped me check the flow by checking if the UI is understandable and if the features match what is needed for the project.